

Date: 20/08/2025

TASK - 5

Implement various searching and sorting operations in python

(a) Library Book Search (Linear Search / Binary Search)

AIM: TO write a python program that allows searching for a book in a list of book titles using linear search and binary search (if the list is sorted).

ALGORITHM:

- Linear Search
1. Start from the first element of the list.
 2. Compare each element with the search key.
 3. If found, return the position.
 4. If not found after checking all elements, return "not found".

Binary Search (for sorted lists)

1. Set $low = 0$ and $high = n - 1$.
2. Find $mid = (low + high) // 2$.
3. If $key == list[mid]$, return position.
4. If $key < list[mid]$, search the left half ($high = mid - 1$).
5. Else, search the right half ($low = mid + 1$).
6. Repeat until found or list is exhausted.

Sample OUTPUT

Linear search RESULT:

Book found at position 2

Sorted Books : ['C', 'HTML', 'Java', 'Python', 'SQL']

Binary search RESULT.

Book found at position 2:

program

Linear search

```
def linear-search (books, key):  
    for i in range (len (books)):  
        if books [i] == key:  
            return i  
  
    return -1
```

Binary search

```
def binary-search (books, key):  
    low, high = 0, len (books) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if books [mid] == key:  
            return mid  
        elif books [mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

main

```
books = [ "python", 'C', "Java", "HTML", "SQL"]  
search = book = "Java"
```

```
print ( "Linear Search Result : ")  
pos = linear-search (books, search - book)  
print ( "Binary Search Result : ")
```

```
if pos != -1:
```

```
    print ( f "Book found at position { pos }")
```

```
else:
```

```
    print ( "Book not found")
```

RESULT:

Thus, the task library book search excited and verified successfully.

20/10/21

TASK - 5(b)

Student Grand organizer (Sorting Bubble/Insertion/selection)

AIM:-

To write a python program that organizes students' grades using different sorting techniques

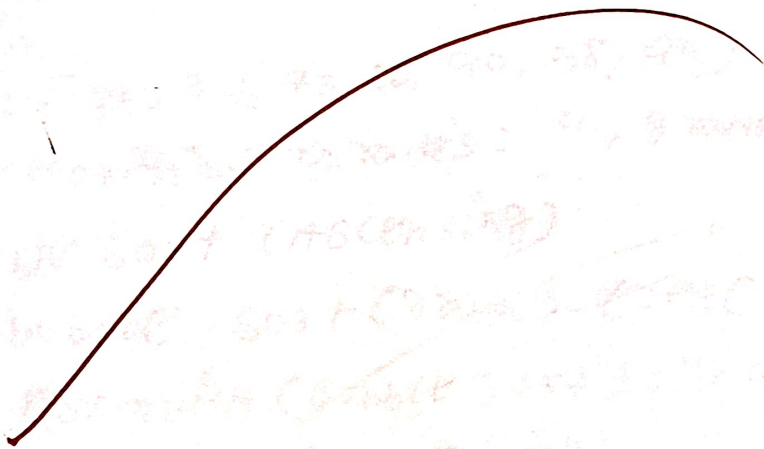
ALGORITHM.

Bubble sort (Ascending)

1. Repeat for $n-1$ passes
2. compare adjacent elements.
3. swap if out of order.
4. After each pass, the largest element moves to the end.

selection sort (Descending)

1. Find the maximum element in the unsorted part.
2. swap with first element of the unsorted part.
3. Repeat until sorted.



SAMPLE OUTPUT

original grades: [85, 92, 75, 66, 90, 58, 99]

Ascending (Bubble Sort): [58, 66, 75, 85, 90, 92, 99]

Descending (Selection Sort): [99, 92, 90, 85, 75, 66, 58]

TOP 3 Scores: [99, 92, 90]

Program.

```
# Bubble sort (Ascending)
```

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n-1):
```

```
        for j in range(n-1-i):
```

```
            if arr[j] > arr[j+1]:
```

```
                arr[j], arr[j+1] =
```

```
arr[j+1], arr[j]
```

```
    return arr
```

```
# Selection sort (Descending)
```

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        max_idx = i
```

```
        for j in range(i+1, n):
```

```
            if arr[j] > arr[max_idx]:
```

```
                max_idx = j
```

```
        arr[i], arr[max_idx] =
```

```
arr[max_idx], arr[i]
```

```
    return arr
```

```
# main
```

```
grades = [86, 92, 75, 66, 90, 58, 99]
```

```
print("original grades: ", grades)
```

```
# Bubble sort (Ascending)
```

```
asc = bubble_sort(grades.copy())
```

```
print("Ascending (Bubble sort): ", asc)
```

```
# Selection sort (Descending)
```

```
desc = selection_sort(grades.copy())
```

```
print("Descending (Selection sort): ", desc)
```

```
# Top 3 scores
```

```
print("Top 3 scores: ", desc[:3])
```

RESULT:

Thus, the task student + Grand Organizer
has existed and verified successfully.

2018